



Agile Manifesto

The Agile Alliance

- The Agile Alliance is a nonprofit organization with global membership, committed to advancing Agile development principles and practices.



- We believe that Agile approaches deliver higher value faster, and make the software industry more productive, humane, and sustainable.

Agile Manifesto

profile of authors at <http://www.agilemanifesto.org/authors.html>

Manifesto for Agile development

We are covering better ways of delivering software by doing it and helping others do it.

Through this work we have come to value:

Individuals & Interactions over processes and tools

Working software over comprehensive Documentation

Customer Collaboration over contract negotiation

Responding to Change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie Van Bennekum
Alistair Cockburn
Warid Cuningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Individuals and Interactions over Processes and Tools...



- People are the most important ingredient of success.
- A good process will not save a project from failure if the team doesn't have strong players, but a bad process can make even the strongest of players ineffective. Even a group of strong players can fail badly if they don't work as a team.

Individuals and Interactions over Processes and Tools...

- A strong player is not necessarily an ace programmer. A strong player may be an average programmer but someone who works well with others.

Working well with others communicating and interacting is more important than raw programming talent.



Individuals and Interactions over Processes and Tools...



- A team of average programmers who communicate well are more likely to succeed than is a group of superstars who fail to interact as a team.

Individuals and Interactions over Processes and Tools...



- The right tools can be very important to success. Compilers, interactive development environments (IDEs), source code control systems, and so on, are all vital to the proper functioning of a team of developers.

Individuals and Interactions over Processes and Tools...



- However, tools can be overemphasized. An overabundance of big, unwieldy tools is just as bad as a lack of tools.

Individuals and Interactions over Processes and Tools...



Our advice is to start small.

Don't assume that you've outgrown a tool until you've tried it and found that you can't use it.

Instead of buying the top-of-the-line, mega expensive source code control system, find a free one and use it until you can demonstrate that you've outgrown it.

Individuals and Interactions over Processes and Tools...

Before you buy team licenses for the best of all computer-aided software engineering (CASE) tools,

use whiteboards and graph paper until you can unambiguously show that you need more.

Use simple time sheets, traceability metrics, meeting agenda

Before you commit to the top-shelf database system, try flat files. Don't assume that bigger and better tools will automatically help you do better. Often, they hinder more than they help.

Individuals and Interactions over Processes and Tools...

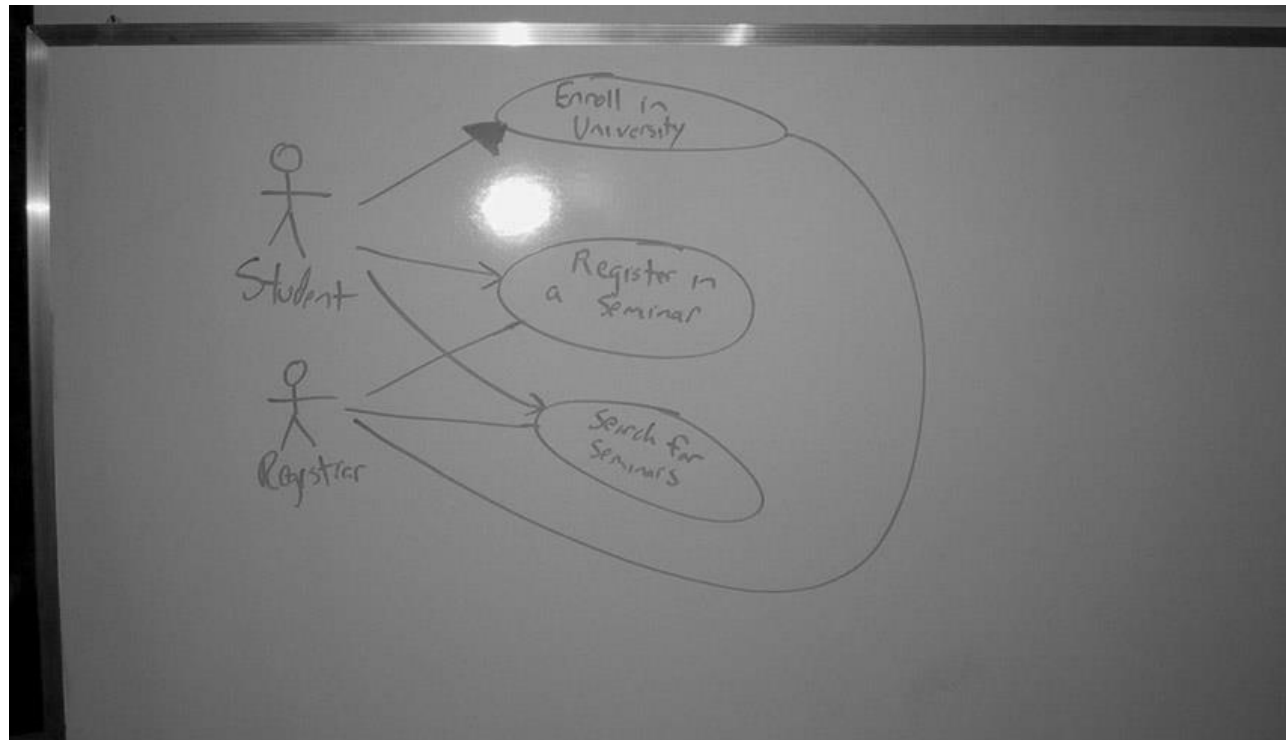
Remember, building the team is more important than building the environment.

Many teams and managers make the mistake of building the environment first and expecting the team to gel automatically. Instead, work to create the team, and then let the team configure the environment on the basis of need.

Activity- place any thing in your office of your choice

Making Whiteboards Work for Software Development

- shows a digital photo taken of a whiteboard sketch of a simple use case diagram, which **I drew with a couple of stakeholders**. We took the photo because we were afraid that we would lose the **valuable information that it contains**,



Working Software over Comprehensive Documentation...



- Software without documentation is a disaster. Code is not the ideal medium for communicating the rationale and structure of a system.
- Rather, the team needs to produce human-readable documents that describe the system and the rationale for design decisions.

Working Software over Comprehensive Documentation...

- However, too much documentation is worse than too little. Huge software documents take a great deal of time to **produce** and even more time to keep in **sync** with the code. If they are not kept in sync, they turn into large, complicated lies and become a significant **source of misdirection**.

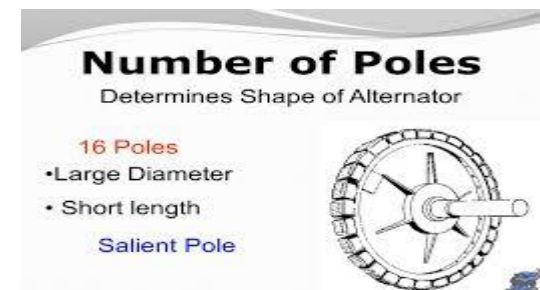
- Case study of Google ara



Working Software over Comprehensive Documentation...

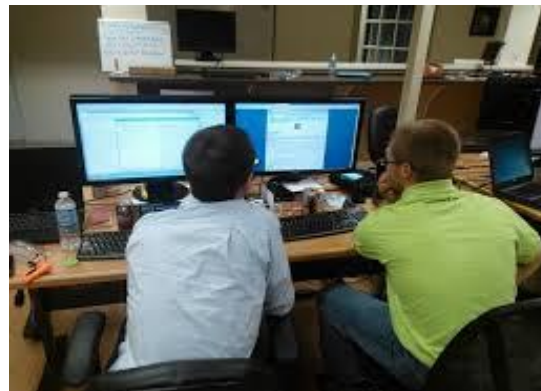
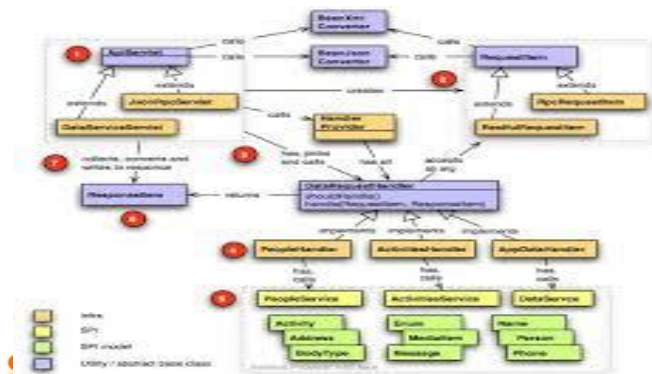
It is always a good idea for the team to write and maintain a short rationale and structure document. But that document needs to be short and salient.

By short, I mean one or two dozen pages at most. By salient, I mean that it should discuss the overall design rationale and only the highest-level structures in the system.



Working Software over Comprehensive Documentation...

- If all we have is a short rationale and structure document, ***how do we train new team members*** about the system? We work closely with them.
- We transfer our knowledge to them by sitting next to them and helping them. We make them part of the team through close training and interaction.



Working Software over Comprehensive Documentation...



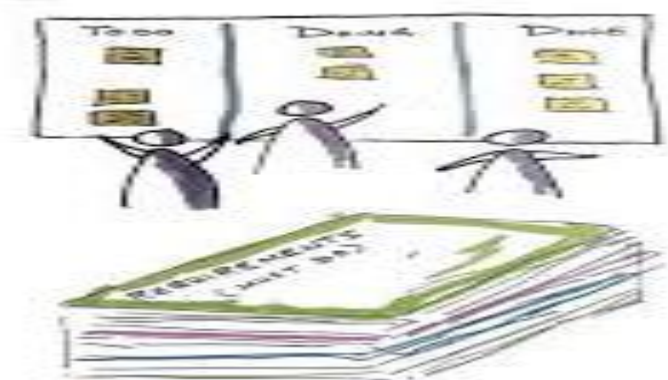
The two documents that are the best at transferring information to new team members are the code and the team.

The code does not lie about what it does. It may be difficult to extract rationale and intent from the code, but the code is the only unambiguous source of information. The team holds the ever-changing roadmap of the system in its members' heads.

The fastest and most efficient way to put that roadmap down on paper and transfer it to others is through human-to-human interaction.

Customer Collaboration over Contract Negotiation...

- **Best contracts are NOT** those specifying requirements, schedule and cost.
 - Become meaningless shortly.
- **Far better are contracts that govern the way the development team and customer will work together.**



Customer Collaboration over Contract Negotiation...

Successful projects involve customer feedback on a regular and frequent basis.

Rather than depending on a contract, or a statement of work, the customer of the software works closely with the development team, providing frequent feedback on its efforts.

Activity: define stakeholder, and possible communication

Responding to Change over Following a Plan.....



- The course of a software project cannot be planned very far into the future.
- First, the business environment is likely to change, causing the requirements to shift.

Responding to Change over Following a Plan.....



- Second, once they see the system start to function, customers are likely to alter the requirements.
- Finally, even if we know what the requirements are and are sure that they won't change, we are not very good at estimating how long it will take to develop them.

Responding to Change over Following a Plan.....

- **As the team gains knowledge about the system and as the customer gains knowledge about the team's needs,** certain tasks on the chart will become unnecessary. Other tasks will be discovered and will need to be added.
- In short, the plan will undergo **changes in shape, not only in dates.**

Responding to Change over Following a Plan.....

A better planning strategy is to make detailed plans for the next week, rough plans for the next 3 months, and extremely crude plans beyond that.

We should know the individual tasks we will be working on for the next week.

We should roughly know the requirements we will be working on for the next 3 months. And we should have only a vague idea what the system will do after a year.

Responding to Change over Following a Plan.....

- This decreasing resolution of the plan means that we are investing in a detailed plan only for those tasks that are immediate.
- Once the detailed plan is made, it is difficult to change, since the team will have a lot of momentum and commitment.
- But since that plan governs only a week's worth of time, the rest of the plan remains flexible.



Agile Manifesto

profile of authors at <http://www.agilemanifesto.org/authors.html>

Discussion

Individuals & Interactions over processes and tools
Working software over comprehensive Documentation
Customer Collaboration over contract negotiation
Responding to Change over following a plan

Agile process and methods

- XP
- Lean
- KanBan
- Crystal
- Feature Driven development
- DSDM
- PRINCE 2
- Microsoft Solutions Framework
- Agile Unified Process (AUP)
- EVO
- Adaptive Software Development
- behavior-driven development (BDD)
- SCRUMBAN
- SCRUM
- Disciplined agile delivery
- NEXUS
- Setchu – Agile at Scale
- Squish (Froglogic)
- Rapid application development (RAD)
- Holistic Software Development
- Large-scale scrum (LeSS), Craig Larman and Bas Vodde